Ollscoil
Teicneolaíochta
an Atlantaigh

Atlantic
Technological
University

AutoSync Car Infotainment System

Project Engineering

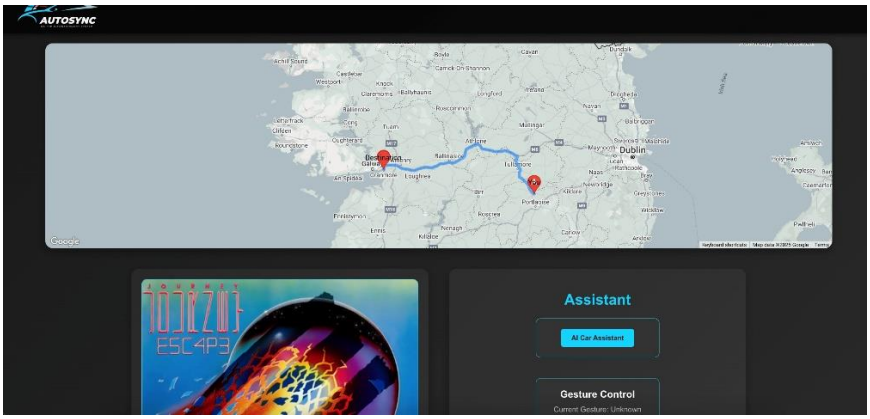
Year 4

Olamide Aderogba

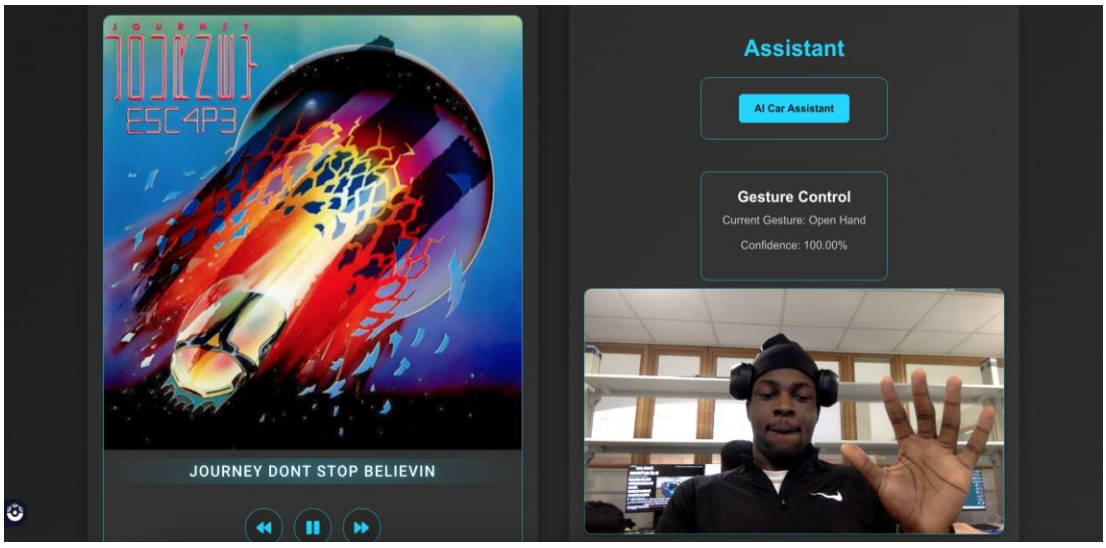
Bachelor of Engineering (Honours) in Software and Electronic Engineering

Atlantic Technological University

2024/2025


AUTOSYNC
Assistant
AI Car Assistant
Gesture Control


Speak
Ready to listen...


JOURNEY DONT STOP BELIEVIN
Assistant
AI Car Assistant
Gesture Control
Current Gesture: Open Hand
Confidence: 100.00%

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Galway-Mayo Institute of Technology.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

Olamide Aderogba_____

Acknowledgements

I would like to acknowledge my immense appreciation to the individuals who supported me on the development of this car infotainment system project from September 2024 until April 2025. First, I would like to thank my first project supervisor, Paul Lennon, for his exceptional supervision and encouragement during the weekly stand ups. I would also like to thank Pat Hurney, my assigned Project supervisor after Christmas, who was able to share his knowledge on Machine Learning with me and guide me throughout my project work, which ultimately influenced the project towards the hands-free vision and guided me to build an understanding framework to negotiate usability issues.

Table of Contents

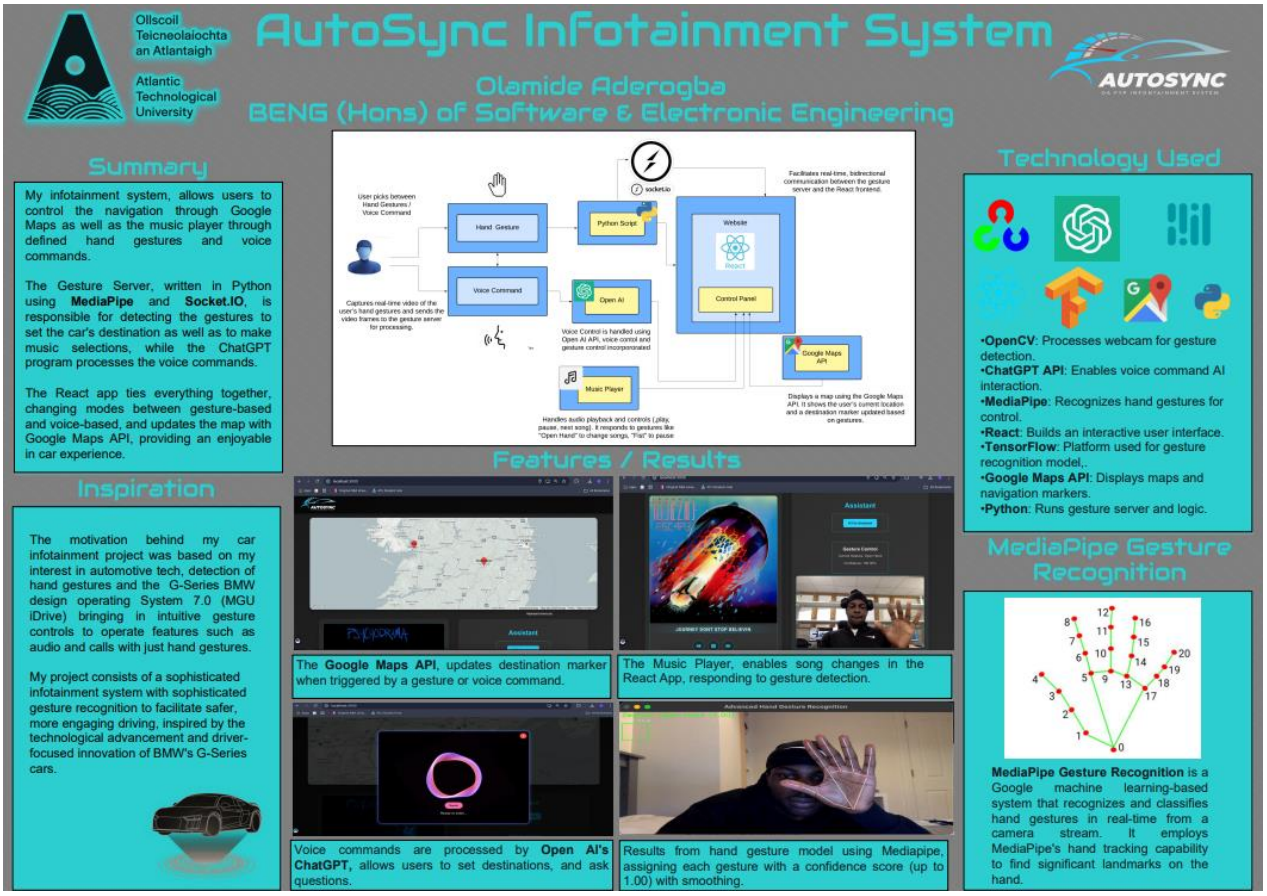
1 Summary

This project was to develop a hands-free car infotainment system, in which driver safety and driver experience can be improved by allowing a user to perform hands free navigation and music playback control through gestures and voice commands. Due to these improvements to safety, the motivation behind the project was to reduce driver distraction, a common factor that contributes to a high number of vehicle accidents. In fact, In Ireland, distracted driving is prominent, with 143 road deaths recorded in 2023 and mobile phone use identified as a contributory factor to collisions (Road Safety Authority, 2023), something that could've been reduced with a system that was not touch based. The project scope was limited and was focused specifically on base functionalities, which included navigation capabilities such as setting destination, looking at, and confirming traffic updates and music functions such as play, pause and adjusting volume.

A client server architecture was employed for modular development. The frontend was developed in React, creating a responsive UI with components for the map, music player, and control panel. The Google Maps API was utilised to allow the map to set destinations and calculate a route. The OpenAI API was used to process voice commands and run on a Node.js backend with Express to help with natural language processing. A Python server was used to detect hand gestures by using OpenCV and MediaPipe, which took input from the camera feed. Gesture recognition processed in real time and all updates were sent to the frontend via Socket.IO (a library for real-time communication). The system successfully mapped hand gestures and voice commands to play music and present navigation updates with an intuitive use case.

During testing, gesture recognition was about 85% accurate, at a 0.7 confidence threshold, and voice commands had a 90% success rate, with navigation and music controlling functioning reliably throughout all test cases. In conclusion, this project demonstrates the viability of hands-free interfaces and how it can be implemented into vehicle through the use of certain applications such as React, the Google Maps API, the OpenAI API, and OpenCV/MediaPipe.

2 Poster


Ollscoil Teicneolaíochta an Atlantaigh
Atlantic Technological University
AutoSync Infotainment System
Olamide Aderogba
BENG (Hons) of Software & Electronic Engineering
AUTOSYNC
Summary
My infotainment system, allows users to control the navigation through Google Maps as well as the music player through defined hand gestures and voice commands.
The Gesture Server, written in Python using MediaPipe and Socket.IO, is responsible for detecting the gestures to set the car's destination as well as to make music selections, while the ChatGPT program processes the voice commands.
The React app ties everything together, changing modes between gesture-based and voice-based, and updates the map with Google Maps API, providing an enjoyable in car experience.
Inspiration
The motivation behind my car infotainment project was based on my interest in automotive tech, detection of hand gestures and the G-Series BMW design operating System 7.0 (MGU iDrive) bringing in intuitive gesture controls to operate features such as audio and calls with just hand gestures.
My project consists of a sophisticated infotainment system with sophisticated gesture recognition to facilitate safer, more engaging driving, inspired by the technological advancement and driver-focused innovation of BMW's G-Series cars.
User picks between Hand Gestures / Voice Command
Hand Gesture
Voice Command
Captures real-time video of the user's hand gestures and sends the video frames to the gesture server for processing.
socket.io
Python Script
Open AI
Website
React
Control Panel
Google Maps API
Music Player
Facilitates real-time, bidirectional communication between the gesture server and the React frontend.
Voice Control is handled using Open AI API, voice control and gesture control incorporated
Displays a map using the Google Maps API. It shows the user's current location and a destination marker updated based on gestures.
Handles audio playback and controls (play, pause, next song). It responds to gestures like "Open Hand" to change songs, "Fist" to pause
Features / Results
The Google Maps API, updates destination marker when triggered by a gesture or voice command.
The Music Player, enables song changes in the React App, responding to gesture detection.
Voice commands are processed by Open AI's ChatGPT, allows users to set destinations, and ask questions.
Results from hand gesture model using Mediapipe, assigning each gesture with a confidence score (up to 1.00) with smoothing.
Technology Used
•OpenCV: Processes webcam for gesture detection.
•ChatGPT API: Enables voice command AI interaction.
•MediaPipe: Recognizes hand gestures for control.
•React: Builds an interactive user interface.
•TensorFlow: Platform used for gesture recognition model,.
•Google Maps API: Displays maps and navigation markers.
•Python: Runs gesture server and logic.
MediaPipe Gesture Recognition
MediaPipe Gesture Recognition is a Google machine learning-based system that recognizes and classifies hand gestures in real-time from a camera stream. It employs MediaPipe's hand tracking capability to find significant landmarks on the hand.

3 Introduction

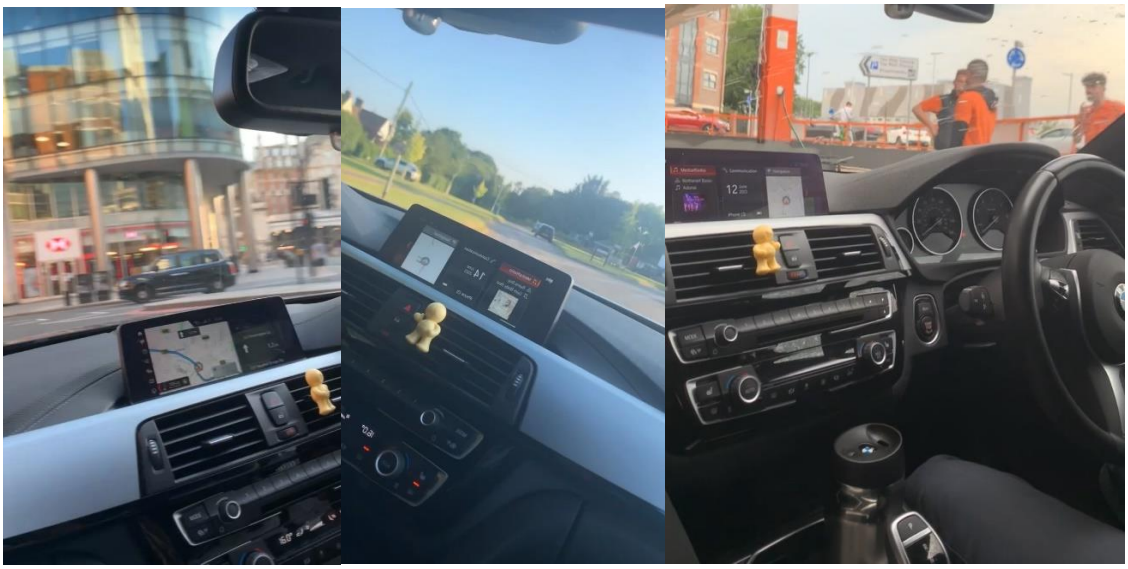
Modern vehicles are now incorporating infotainment systems more frequently than ever before. This has drastically changed the way we drive, giving users access to navigation tools, entertainment, and communication tools. Distraction while driving is a danger in Ireland, as 28% of drivers admitted to using their phone while driving, resulting in road crashes[19]. Together with the 8.1% of fatal car crashes caused by distraction in the US [1] this shows there is a global need for safer infotainment systems. The aim of my project was to develop a car infotainment system that made use of hand gestures to control navigation and control of music playback so drivers can keep eyes on the road and reduce distraction caused crashes in Ireland and beyond. This will reduce the amount of manual driving involvement while also enhancing the safety of the driver.

The motivation for this work is the increasing demand for safer in-vehicle interfaces that will allow people to keep their attention on the road. Gesture and voice control represent an intuitive interface that takes advantage of developments in computer vision [9] and natural language processing [3], which are part of creating a natural user experience. Gesture and voice control fit with the direction the automotive industry is moving, by recognising the potential for new hands-free technologies to help improve safety and user enjoyment [5].

The scope is limited to navigation and music control, adding gesture-based activities and voice command activities along with live traffic updates using the Google Maps API [2], excluding phone call capabilities. The report structure will view the project from the development perspective starting with an overview then system architecture, design and implementation, testing, challenges, and finally conclude with findings and recommendations for future work.

4 Background

From a young age, I became fascinated with technology through my interest and love for music and how it relates to new technologies. I immersed myself in countless hours of research about music playback technologies and my interest shifted to investigating gesture control and Artificial Intelligence (AI) systems to facilitate better engagement experiences. My idea for the AutoSync Car Infotainment System happened while I was a passenger in my uncle's BMW 3 Series (G20) in Summer 2023, it had an incredible gesture control feature that allowed me to skip and adjust the volume of the music just by waving my hand, and other functionalities through other gestures [11].



I thought it would be an excellent way to minimise driver distraction, but I also found it limiting, it had limited flexibility and no voice functionality. Therefore, I wanted to create an entirely hands-free system that integrated gesture control with AI enabled voice recognition using systems for navigation and music playback. The need for my project is evident as distracted driving caused 143 road deaths in Ireland in 2023 [20] and 8.1% of fatal crashes in the US [1], which demonstrates a growing use for safer in-vehicle interfaces.

4.1 Evolution of Hands-Free Interaction in Car Infotainment Systems

Car infotainment systems have progressed from basic radios to sophisticated interfaces that merge navigation and entertainment systems. The BMW 2001 iDrive was a key innovation that

centralised navigation and audio management, paving the way for hands-free interfaces [11]. The increase in distracted driving, causing 8.1% of fatal crashes in 2023 [1], necessitated gesture and voice controls to enhance driver safety. In 2015, BMW introduced the gesture control feature on the 7 Series, enabling the driver to adjust volume or accept calls with a wave of the hand, and later released the Intelligent Personal Assistant device in 2018, allowing for voice commands to find a gas station.

My project establishes this platform and investigates gesture and voice controls for navigation and for music playback. My approach can easily be expanded to other components of car systems by adding worthwhile, affordable hardware, like cameras and microphones, into potential open-source systems such as Android Automotive OS. This expands hands-free interaction possibilities across all segments of vehicles, while enhancing sustainability.

4.2 Effectiveness of Hands Free Control

My important research question was how well gesture and voice control mitigate driver distraction. Research indicates touchscreen use increases visual demand, while hands-free options allow users to keep their eyes on the road [5]. BMW's "hands on the wheel, eyes on the road" design philosophy fits well with the designed functions for my project where the user can control volume with a simple thumbs up gesture, and use voice commands to minimise manual interaction [11].

5 Project Architecture

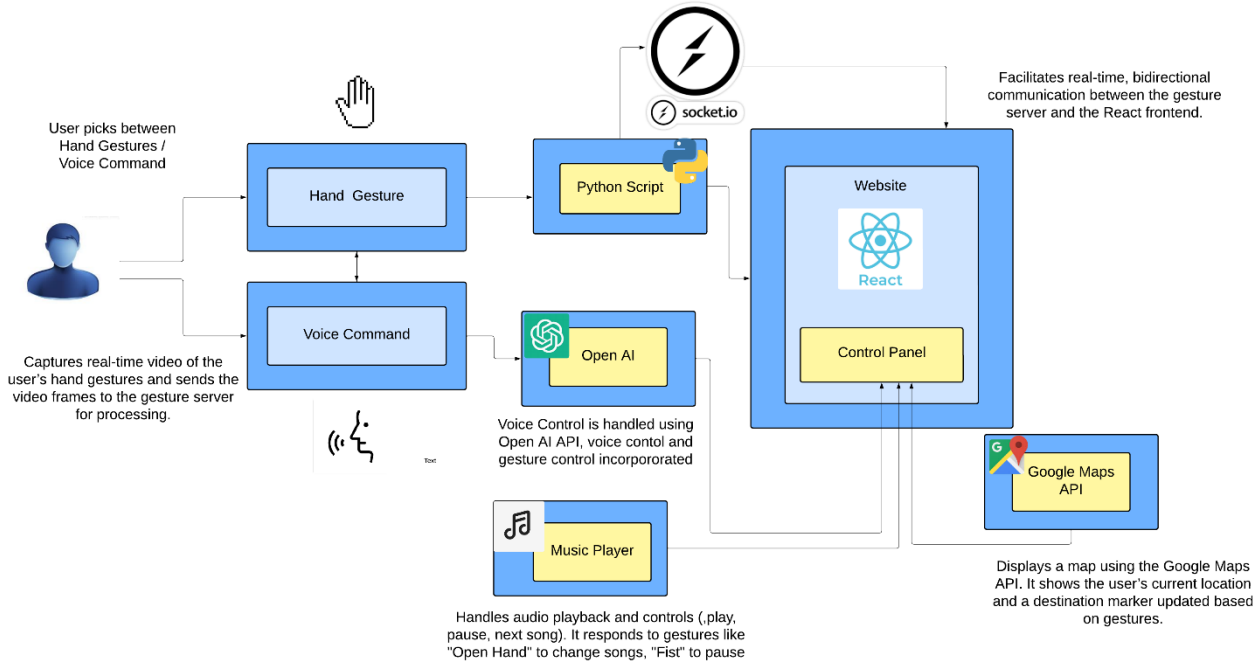


Figure 5-1 Architecture Diagram

6 Project Plan

The design of the car infotainment system with gesture and voice control was undertaken over a total of 24 weeks of active work divided over two semesters separated by a break. Weeks-1-11 were completed between September 26, 2024 and December 15, 2024 and Weeks-12-24 were completed between January 21, 2025 and April 15, 2025. The project was managed using Jira sprints that allowed me to effectively manage the project work, a sprint took place every two weeks. This section includes objectives and a detailed timeline week-by-week, with milestones, tasks, and resources to serve as an all-inclusive schedule of everything that occurred in the project across the timeline.

6.1 Objectives and Scope Recap

The main goal was to create a car infotainment system that allowed groups of people driving, to use gestures and voice commands to operate navigation and music playback systems hands-free while driving, ultimately minimising distraction to the driver and increasing safety. Some specific objectives of the project included integrating navigation tools within mapping systems, deploying the Google Maps API, ensuring gesture recognition was accurate, and using appropriate methods to guarantee reliable voice command processing. The focus of the project was based on navigation (setting destinations, checking current traffic situations) and music playback (play/pause, volume), and did not include operating additional features in any phone call application, or an ability to view any maps outside of the specific navigation area and protocols outlined.

6.2 Timeline and Milestones

The project spanned exactly 24 weeks of active work, divided into 12 two-week Jira sprints. The first semester covered Weeks 1–11 (Sep. 26–Dec. 15, 2024). The second semester covered Weeks 12–24 (Jan. 21–Apr. 15, 2025). The initial focus was more focused on a gestured/voice command music player through the use of Spotify API, but after the Spotify API ban on November 27, 2024, the project pivoted to a car infotainment system in January 2025. Below is the detailed timeline with key milestones:

Sprint 1 (Weeks 1–2, Sep. 26–Oct. 9, 2024): Researched the Spotify API and React for user interface development. Started configuring the Spotify API to allow user authentication, having issues with set up with the redirection URI.

Sprint 2 (Weeks 3–4, Oct. 10–Oct. 23, 2024): Configured the Spotify API redirection URI and was able to resolve the OAuth issues I had been facing. Started to develop React user interface components for Spotify playback.

Sprint 3 (Weeks 5–6, Oct. 24–Nov. 6, 2024): Worked on the ESP32-S3 ESP-IDF Skainet environment to implement voice commands and was able to get the ESP32-S3 operational.

Sprint 4 (Weeks 7–8, Nov. 7–Nov. 20, 2024): Researched Spotify UI prototypes with playlists and made some advances setting up the ESP32-S3 voice recognition.

Sprint 5 (Weeks 9–10, Nov. 21–Dec. 4, 2024): Spotify API ban happened on Nov. 27. This started to involve pivot planning.

Sprint 6 (Weeks 11–11, Dec. 5–Dec. 15, 2024): After the potential ban in the Spotify API, pivot options were assessed and collectively abandoned the ESP32-S3 voice approach due to pivot requirements and inconsistency in performance. Established the first level of requirements for car infotainment system.

Break (Dec. 16, 2024–Jan. 20, 2025): No action on project work over the semester break due to exams.

Sprint 7 (Weeks 12-14, January 21 - February 3, 2025): Completed pivot to a Car Infotainment system with navigation and local music playback. Tested gesture recognition with TensorFlow Object Detection but ran into version compatibility issues.

Sprint 8 (Weeks 14-15, February 4 - February 17, 2025): Switched to MediaPipe landmarks for gesture recognition after TensorFlow issues. Completed research of Google Maps API and playback local music.

Sprint 9 (Weeks 16-17, February 18- March 3, 2025): Created React frontend components (GoogleMapComponent, MusicPlayer, ControlPanel) and stablished Node.js backend with OpenAI API for voice commands.

Sprint 10 (Weeks 18-19, March 4- March 17, 2025): Implemented Python gesture recognition using OpenCV/MediaPipe, connected it to backend with Socket.IO.

Sprint 11 (Weeks 20-21, March 18- March 31, 2025): Constructed voice control using AIInterface, and completed system integration testing including API and Socket.IO communication.

Sprint 12 (Weeks 22 -24, April 1- April 15, 2025): Conducted unit tests with music playback, integration tests voice-commands to turn on navigation, and user testing. (Milestone: Final evaluation completed, Apr. 19).

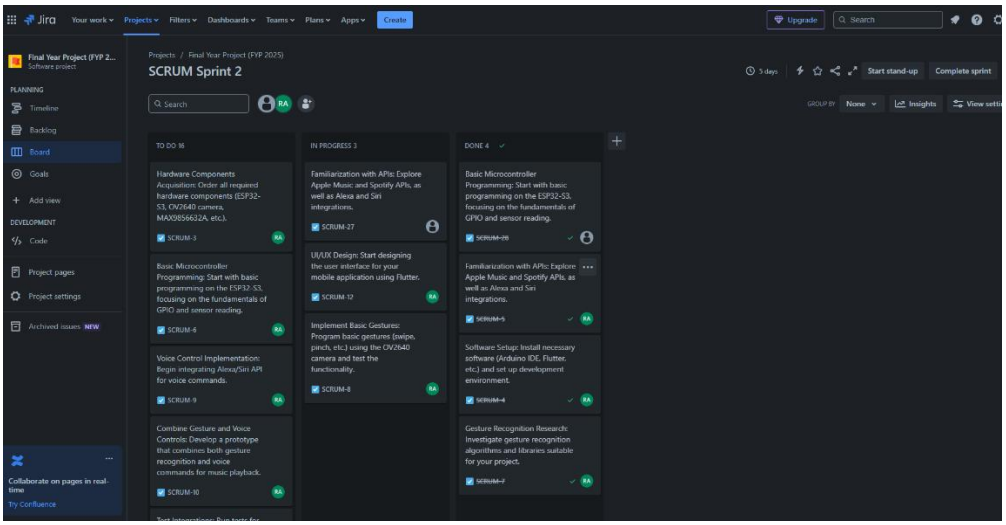


Figure 6-2 Jira Sprint Board

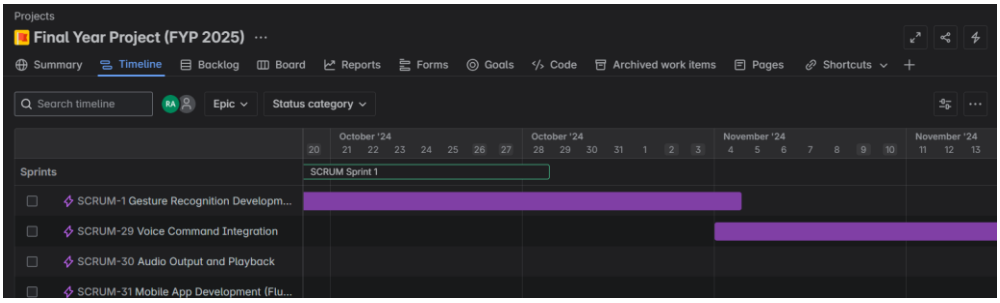


Figure 6-3 Timeline Jira

Figure 6-4 Burn Down Chart

Figure 6-5 Burn Down Chart

7 System Architecture

The car infotainment system utilising gesture and voice control was developed with a modular client-server architecture for scalability and interaction among components. It consists of three main components: the client-side user interface built with React, the backend built with Node.js and Express to process voice command requests, and the gesture recognition server built with Python to detect hand gestures in real-time. The frontend user interface, hosted on the client-side, is responsible for displaying the map, music player, and control panel; which receives user input via gesture and voice command. The Node.js server processes voice commands through the OpenAI API by communicating with the client-side and the gesture recognition server, the latter communicates with the client-side via Socket.IO wherein Python processes visualisation results through OpenCV and MediaPipe to facilitate gesture detection and update the relevant changes to the front end. The system also includes navigation functionalities through the Google Maps API. This architecture, depicted in the block diagram above in (Figure 5-1) enables seamless flow of data and response time across all components.

System Architecture Overview

This car infotainment system is designed with a client-server model:

- Frontend: A React app, the UI that integrates Google Maps for navigation as well as gesture/voice control and a music player.
- Backend: A Python script (gesture_server.py) that will run in a real-time process using a webcam for gesture recognition.
- Communication: The frontend and backend communicate through WebSocket (Socket.IO).

App.js: Root component that controls state (destination, currentLocation, theme.

GoogleMapComponent.js: Renders google map, controls navigation (DirectionsRenderer); traffic layer (TrafficLayer); displays POI markers.

ControlPanel.js: Manages gestures, voice control, and music player; shows toast notifications.

GestureControl.js: Accepts gesture data via WebSocket, renders gesture and camera feed.

VoiceControl.js: Accepts voice input with web speech API; renders AI overlay.

MusicPlayer.js: Manages music playback with gestures (notably; "Open Hand" command to play/pause.

Backend (Gesture Recognition Server)

gesture_server.py: Processes webcam video, recognises gestures with MediaPipe, sends data via WebSocket.

- Key Functions: calculate_distance, calculate_angle, normalize_distance, get_hand_orientation, is_finger_extended, recognize_gesture, smooth_gesture.

Figure 7 Javascript and CSS files for project

Technologies Used

Frontend

- **React(v2.19.3)**: UI framework.

  Why I chose it: I chose React because of its component reusability and effective state management capabilities streamlined with my modular UI (map, music player) [7]. I chose it over Angular for its smaller footprint and faster learning curve, and over Vue.js for its stronger community support and ecosystem, helping to ensure better availability of resources and libraries.

- **@react-google-maps/api (v2.19.3)**: Google Maps integration.

  Why I chose it: I chose this library for its easy integration with React, so I can render maps and manage the state of the maps (markers, traffic layers) from components [2]. I picked it over the raw Google Maps API to save boilerplate and over Leaflet for its direct compatibility with any Google Maps features that were critical to my navigation system.

- **Google Maps JavaScript API**: Map rendering, navigation, traffic.

  Why I chose it: I chose this API because it has strong mapping features, real-time traffic information, and navigation capabilities that were needed for my in-vehicle system [23]. I preferred this API over the OpenStreetMap API because it is more accurate and covers all parts of the world--meaning users can reliably navigate from place to place.

- **Google Maps Places API**: Nearby place search.

  Why I chose it: I added this API to allow place searches (e.g., looking for gas stations around a user) to increase functionality[2]. I chose this API over other APIs, like HERE Places API, because it is native to Google Maps JavaScript API, which simplifies the integration process, and because there is better documentation available to implement it.

- **Socket.IO Client (v4.7)**: WebSocket communication.

  Why I chose it: I chose Socket.IO for how it can transmit gesture updates in real time from the frontend all the way to the backend, which is key to providing responsive and hands-free control [8]. I chose it over WebRTC because it has less set up, and I chose it over using plain WebSocket's because it comes with a lot of useful features such as automatic reconnection handling, which in a driving context gave me piece of mind.

- **Web Speech API**: Voice recognition.

Why I chose it: I used this API to facilitate voice command recognition and feedback that allows for hands-free interaction. I used this over third-party libraries such as Speechly because it is natively supported in browsers and does not add a variance of dependencies, it also gave me enough accuracy for my use case without any extra cost.

- **Geolocation API**: User location.

  Why I chose it: I used this API to get the user's current position for the navigation feature, which is more or less the primary feature of my system. I picked this one, instead of an IP service, because it is typically more accurate through GPS, it has no third party dependencies, and it offers live location data with minimal latency.

- **CSS**: Styling with themes.

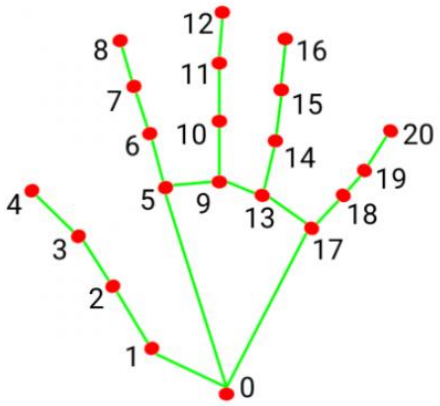
  Why I chose it: Made it look nice using CSS with some themes to provide visual clarity (pulsing animations to confirm voice statements), bought into CSS rather than CSS frameworks such as bootstrap to control styles fully and reduce bundle size, and it was easier than writing all of the CSS in SCSS for what I needed which was basic/straightforward.

Backend

- **Python (v3.12)**: Scripting language.

  Why I chose it: I chose Python because it offers tremendous libraries in computer vision. For example, MediaPipe and OpenCV are well-known resources that are perfect for helping build my gesture recognition script [15]. I wanted to use Python instead of C++ (Python is easier to use and can be developed more quickly), and I wanted to use Python instead of Java as Java syntax is more difficult, and there is more support for machine learning files.

- **MediaPipe (v0.10.14)**: Hand tracking and gesture recognition.



0. WRIST
1. THUMB_CMC
2. THUMB_MCP
3. THUMB_IP
4. THUMB_TIP
5. INDEX_FINGER_MCP
6. INDEX_FINGER_PIP
7. INDEX_FINGER_DIP
8. INDEX_FINGER_TIP
9. MIDDLE_FINGER_MCP
10. MIDDLE_FINGER_PIP
11. MIDDLE_FINGER_DIP
12. MIDDLE_FINGER_TIP
13. RING_FINGER_MCP
14. RING_FINGER_PIP
15. RING_FINGER_DIP
16. RING_FINGER_TIP
17. PINKY_MCP
18. PINKY_PIP
19. PINKY_DIP
20. PINKY_TIP

Why I chose it: I selected MediaPipe because of its accuracy for hand tracking and low latency, which is critical for gesture control in real-time [4]. I chose MediaPipe over TensorFlow.js because it was lightweight and performed well on computationally limited devices. I chose MediaPipe over OpenPose, because it was easier to integrate into my own project using Python and also used pre-trained models.

- **OpenCV (v4.10)**: Video capture and processing.

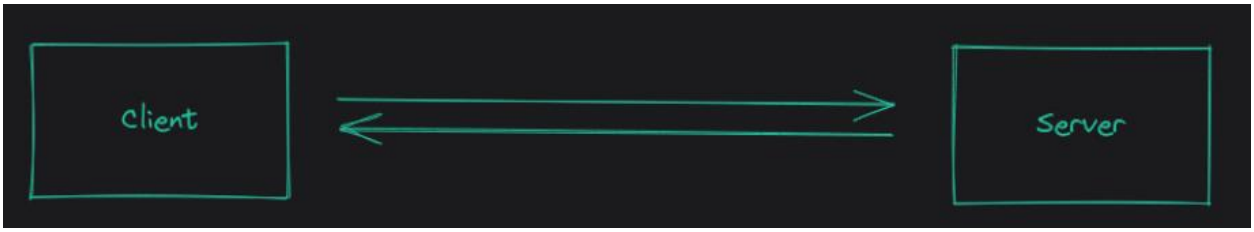
  Why I chose it: I was lucky to use OpenCV as the video frame capture and processing tool for gesture recognition as it is a powerful image processing tool [9]. I figured it would be better than SimpleCV because it has a larger community and better than MATLAB which costs money, mainly because it is free and compatible with Python.

- **NumPy (v2.1.1)**: Numerical computations.

  Why I chose it: The reason I included NumPy to use for my gesture recognition script for handling array operations, such as handling coordinates for a hand's landmarks, is because I used it instead of SciPy since it was plural purposes focused on array manipulation rather than just a wider array of functions [21]. I also used it instead of Pandas since it was much less complex than Pandas, and I didn't need large data manipulation.

- **Socket.IO (Python, v3.0)**: WebSocket server.

  Why I chose it: I utilised Socket.IO to enable me to maintain real-time communication between my Python script and Node.js server sending gesture updates to the frontend. I used Socket.IO for various reasons [8], it worked well with my JavaScript Socket.IO client, and I liked it better than Flask-SocketIO, and it came with reconnections built in, which was nice to have for reliability, rather than relying on WebSocket's where I would need to build a reconnect.



- **Base64**, **Collections (deque)**, **Logging**: Python utilities.

  I used Base64 to encode the video frames for the task of transmission, used deque for efficient buffering of gesture data, and logging for debugging. What I chose to fit the purpose better than alternatives when I consider things like performance, and structured output, I didn't look at any other options when implementing my system.

Development Tools

- **Node.js (v18.20)**, **npm (v10.8.3)**: React runtime and package manager.
- **dotenv (v16.4)**: Environment variables.

7.1 Navigation System

The navigation system is one of the core capabilities of the car infotainment system that provides drivers with another mechanism to manage their travel while keeping their hands free in order to minimise manual processes and enhance safety. In this navigation module, drivers have the flexibility to set a destination point by voice or by gesture, view and receive updates on traffic conditions, and get recommendations on travel routes. This provides users with flexible options to determine a destination point. After selecting the destination point, the navigation system updates the map continuously as real-time information to identify areas of congestion using the map highlight identified in red, as well as recommend routes by providing alternatives to auto route around congested areas for minor delays.

The GoogleMapComponent on the React frontend takes responsibility for the rendering and management of the navigation interface. The component employs the @react-google-maps/api library, which uses the Google Maps API to provide features like map rendering, tracking the user's location, and plotting routes on the map. The GoogleMapComponent, when mounted, retrieves the user's current location with the browser's Geolocation API (using a default location if declined access permission) and fits and centres the map to that location. The GoogleMapComponent listens for updates on the selected destination from verbal or integrated gesture inputs, and at that time, it calls the API to obtain the route where it displays the route on the user interface. The component listens for directions or alternate routes and display those routes, as well as toggle the traffic layer based on user input to maintain situational awareness. Below is a screenshot of the map interface where map is rendered, along with a route to Galway.

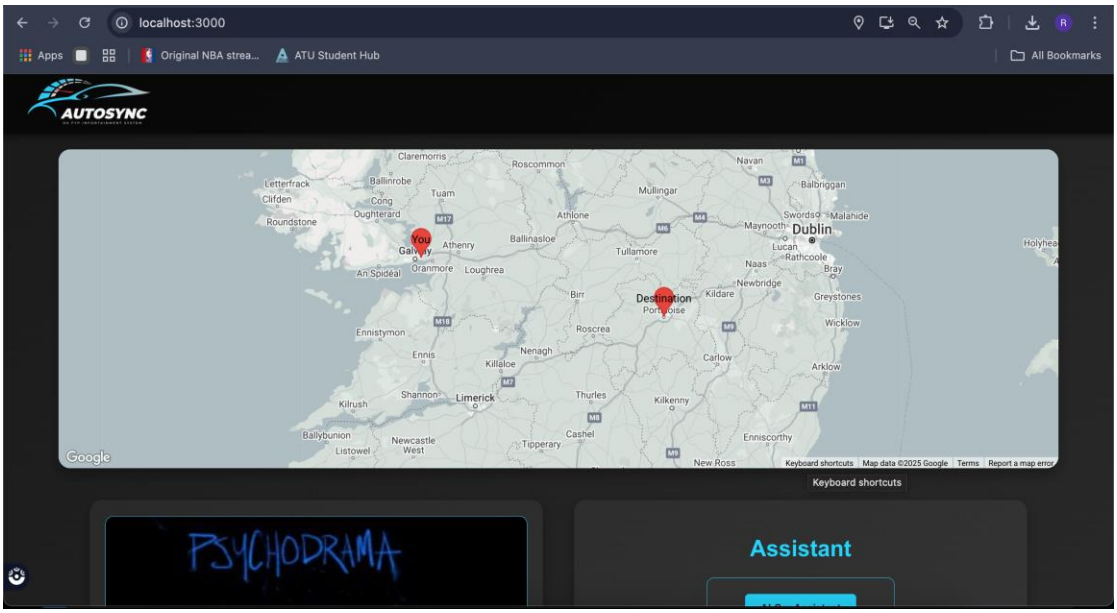


Figure 7-1 Setting Destination POI

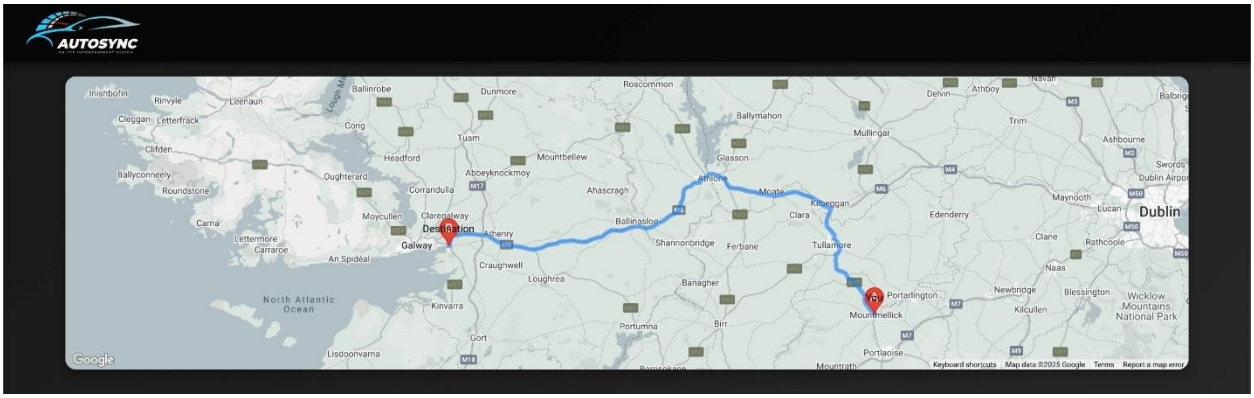


Figure 7-2 Route to Point of Interest

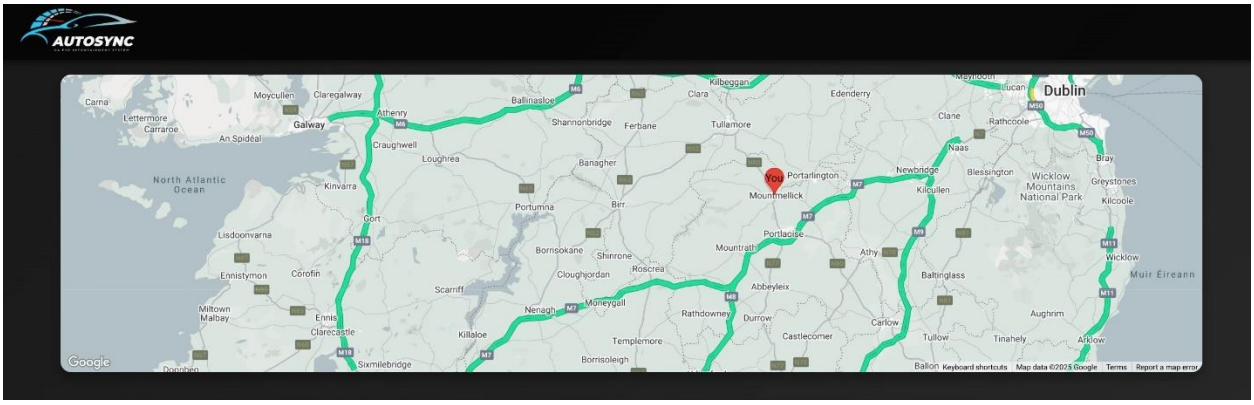


Figure 7-3 Displaying Traffic Congestion

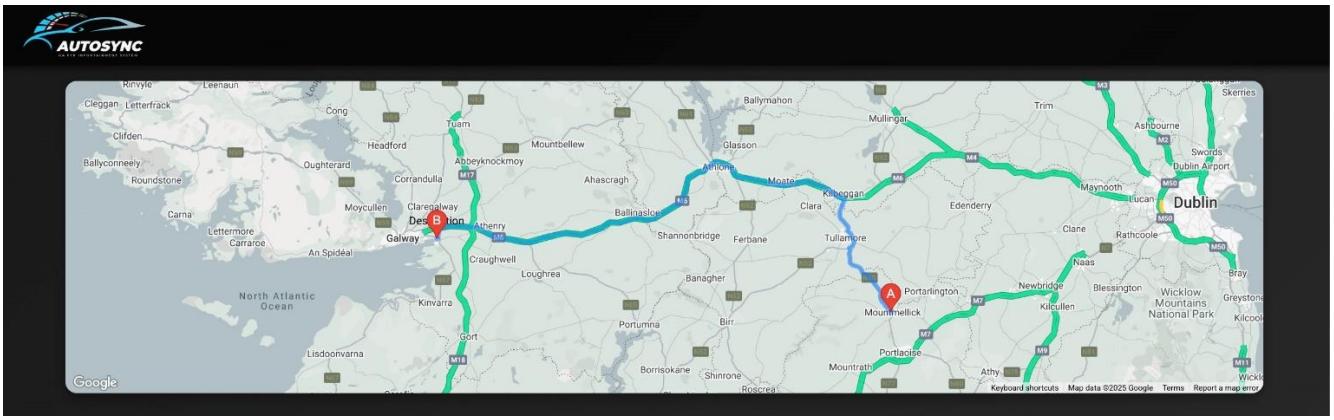


Figure 7-4 Applying Traffic congestion alongside route to Point of Interest

7.2 Music Playback System

The music playback feature is an important functionality of the vehicle's infotainment system that provides drivers the ability to control music playback without having to use their hands, decreasing distractions and improving the driving experience. This module allows users to perform all elements of control, including playing, pausing and adjusting volume via both gestures and voice commands, so that users can choose whichever method is easiest for them. For example, the driver could say “play music” to start or use the "Open Hand" gesture to skip to the next track, and the "Thumbs Up" gesture will adjust volume. If the driver wants to pause the music, they could say “pause” or use the “Closed Fist” gesture. There are multiple control mechanisms for music playback to minimise distractions and allow the driver to keep attention on the road. And in order to maintain a seamless audio experience, the system will maintain the state of music audio even when navigating between music and navigation functions.

The original project sought to connect with Spotify's API to stream music, but that changed after the API ban on November 27, 2024, which led to the use of local music files instead. An

existing library of MP3 files, which were stored locally and fulfilled this function. The React MusicPlayer component was responsible for the playback functionality using the HTML5 element to stream the audio and control playback functionality. The component also kept track of the status of the playback (playing/paused; which song was currently playing; volume level) and would update the UI in real-time to create a dynamic and responsive user experience. Additionally, the React MusicPlayer would listen to gesture and voice movements and map those to actions (playing, paused, skipped) inside an event handler embedded in the other parts of the system.

The music player's UI design is functional and visually pleasing, resulting in an engaging user experience. At the centre of the design is the information about the current song, including the title, artist, and album art. Control buttons (play, pause, skip ahead, skip back) are located below the song title to allow for manual operation if needed. Below the song title and control buttons, there is adjustable volume buttons available and the MusicPlayer also utilises simple gestures to increase or decrease the volume. The button provides clear visual feedback for the user. The MusicPlayer component makes use of the gesture system via Socket.IO updates so that any action taken is reflected in the MusicPlayer UI in real-time. In the screenshot below, the interface for the music player shows the current song “Don’t Stop Believing” by Journey, featuring the album art, title, and the control buttons.

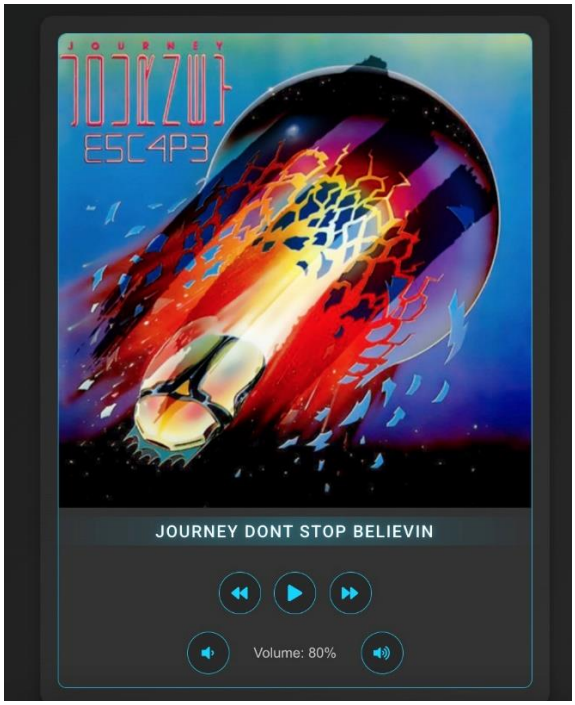


Figure 7-1 Music Card playing current song – Journey Don't Stop Believin

7.3 Gesture Control System

The gesture control function is a central aspect of a car infotainment system, allowing drivers to use hand gestures to control navigation and music playback, reducing the amount of physical interaction needed, and enabling safer driving. This module maps hand gestures to actions, including a "Thumbs up" gesture to increase music volume, an "Open Hand" gesture to skip a music track, and an "L shape" gesture to set a navigation destination and many more. These mappings enable drivers to execute tasks without looking away from the road, congruent with the project goal of reducing distraction. The gesture control module sends the gesture data to the system in real-time, therefore actions are executed immediately. There are multiple predefined gestures that this module can understand, which allows for variation and flexibility depending on the user or action in the infotainment system.

At the start, I experimented with gesture recognition utilising a custom-trained dataset by labelling hand images for gestures such as "Thumbs Up" and "Open Hand" for a machine learning model to then provide the model with data for training. Ultimately, the project suffered from extreme overfitting since the model trained well on the data set but failed to

generalise beyond the data it was trained on, resulting in approximately 90% accuracy which was unrealistic and while running the script the gesture detection was not reflecting 90% accuracy. In January 2025, my focus shifted towards using MediaPipe landmarks which was my initial approach before the Christmas break, to provide a more robust solution, using pre-trained models for hand tracking. The Python script used OpenCV and MediaPipe hand tracking to detect hand landmarks from a camera feed to identify notable points associated with gestures, such as fingertips and wrist, to calculate angles and distances for calculating gestures. The updates are sent to the frontend in real-time using Socket.IO which allows for seamless integration with the rest of the system.

The Python program captures video frames at 30 FPS. Each frame is encoded as a base64 string that is sent to the frontend of the application with bits of gesture data such as the gesture name and confidence. The gesture recognising component of my pipeline captures video (cv2.VideoCapture) and captures frames from the video and uses MediaPipe's Hands solution (mp_hands.Hands) to process the frames. The recognised data is passed through recognise_gesture function, which identifies the gesture and pass through the gesture name and confidence value. The currently recognised gesture will be further processed by a function that smooths the output of detected gestures; this ensures any non-deterministic gestures will pro not be falsely recognised as gestures.

The entire gesture recognition pipeline consists of the flow of: Camera → MediaPipe → Socket.IO → Frontend. The GestureControl component, created with React, receives the updates from the camera operating system and transmit the raw camera footage live to the application with the gesture status like ("Thumbs Up: 0.92 confidence") in the UI panel assigned to it. The GestureControl component uses the useEffect hook to create a Socket.IO connection connected to the web server, where the live camera footage is displayed in base64 style frames and the UI is updated in real-time to show the recognised gestures, providing visual feedback to the user.

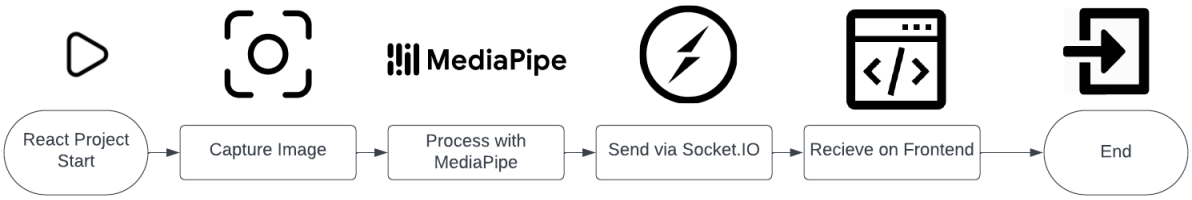


Figure 7-3 Gesture recognition process

7.3.1 Gesture Recognition Functions

The implementation of a Python based gesture recognition system could not have been accomplish without the continued guidance and assistance from ChatGPT[12]. ChatGPT was invaluable in expediting the programming of the gesture recognition system, recommending and implementing complex computer vision strategies, and suggesting techniques for debugging that I ran into as I developed the code. The inspiration for this was drawn from a vision system BMW implemented in their automotive products, which could use 3D detection systems to interpret drivers hand poses within a controlled use case of selecting and manipulating vehicle settings. My ultimate goal was to replicate some of that really amazing 3D detection that enabled the driver to use their hands.

```
def smooth_gesture(current_gesture, confidence):
    global last_gesture
    gesture_history.append((current_gesture, confidence))
    if len(gesture_history) < 2:
        return current_gesture, confidence
    gesture_counts = {}
    total_conf = 0
    for gest, conf in gesture_history:
        gesture_counts[gest] = gesture_counts.get(gest, 0) + conf
        total_conf += conf
    if not gesture_counts or total_conf == 0:
        last_gesture = "Unknown"
        return "Unknown", 0.0
    smoothed_gesture = max(gesture_counts, key=gesture_counts.get)
    smoothed_conf = gesture_counts[smoothed_gesture] / total_conf
```

This code uses a smoothing technique which allows the computer to better recognise hand gestures, because it looks at past history instead of the current guess alone. Given a new gesture guess, and confidence score which is a number between 0 and 1, the code simply adds

them to gesture_history. If there are not at least two guesses in gesture_history, the system simply uses the latest guess. If there are at least two guesses, then it takes all gestures in the history and record the total confidence for each gesture. Whichever gesture has the highest total confidence value. The final confidence is calculated by dividing the winning gestures score by total scores for all conventions in gesture_history. If there were no guesses, or total confidence is zero, code returns "Unknown" with 0% confidence.

```
def get_hand_orientation(landmarks):
    wrist = landmarks.landmark[WRIST]
    middle_mcp = landmarks.landmark[MCP_IDS[2]]
    vector = np.array([middle_mcp.x - wrist.x, middle_mcp.y - wrist.y])
    vertical = np.array([0, -1])
    angle = calculate_angle(wrist, middle_mcp, landmarks.landmark[MCP_IDS[2]])
    return np.degrees(np.arccos(np.dot(vector, vertical) / (np.linalg.norm(vector) or 1)))
```

The get_hand_orientation function computes the hand orientation angle in degrees using hand landmarks, specifically the wrist and the (MCP) joint of the middle finger, to find out how the hand is tilted regarding the vertical axes. The input is a landmarks object with 2D coordinates of hand keypoints. The get_hand_orientation function gets the wrist and the middle MCP joint, a 2D vector was created by taking the middle MCP coordinates (middle_mcp.x, middle_mcp.y) and subtracting the wrist coordinates (wrist.x, wrist.y), to create a vector ([middle_mcp.x - wrist.x, middle_mcp.y - wrist.y]) [20]. This vector, points from the wrist to the middle MCP joint. I also defined a reference vertical vector ([0,-1]), which is the upward direction in a normal image coordinate frame where the vertical axis points downward.

The function then calculates the angle between the two vectors with the dot product definition

$$np.arccos(np.dot(vector, vertical) \ / \ (np.linalg.norm(vector) \ or \ 1))$$

This angle is then converted to degrees with np.degrees to return the vector's orientation with respect to the vertical axis. It returns the orientation to degrees, e.g. - 0 degrees if the hand is perfectly vertical, 90 degrees if horizontal, or 180 degrees if the hand is upside down.

```
def normalize_distance(landmarks, dist):
    wrist = landmarks.landmark[WRIST]
    middle_mcp = landmarks.landmark[MCP_IDS[2]]
    hand_size = calculate_distance(wrist, middle_mcp)
    return dist / hand_size if hand_size > 0 else dist
```

The normalize_distance function determines a normalised distance between two hand landmarks by taking the given dist and dividing it by the size of the hand to obtain scale invariance. The wrist (landmarks.landmark[WRIST]) and the middle finger MCP joint (landmarks.landmark[MCP_IDS[2]]) are extracted and the size of the hand (size_hand) is determined by the Euclidean distance between these two points[20] :

$$\sqrt{(middle_mcp.x - wrist.x)^2 + (middle_mcp.y - wrist.y)^2}$$

The function returns dist/size_hand if size_hand > 0 or dist for all other cases to prevent dividing by zero. The normalisation helps to ensure that the distances are relative to the size of the hand so distance gesture detection is not liable to changes in hand size or camera distance.

```
def calculate_distance(p1, p2):
    return np.sqrt((p1.x - p2.x)**2 + (p1.y - p2.y)**2 + (p1.z - p2.z)**2)
```

The calculate_distance function takes 2 3D points p1 and p2 with their x, y, and z coordinates and calculates the 3D Euclidean distance between the two points. It does this by using the equation:

$$\sqrt{(p1.x - p2.x)^2 + (p1.y - p2.y)^2 + (p1.z - p2.z)^2}$$

The coordinate differences between p1 and p2 for x, y, and z are square, added together, and square rooted with NumPy's np.sqrt. This distance represents the straight line distance between the two points in 3D space. Calculating Euclidean distances is common in applications like gesture recognition as it is used for measuring the spatial relationships between hand landmarks.

7.4 Voice Control System

The gesture control function is a central aspect of a car infotainment system, allowing drivers to use hand gestures to control navigation and music playback, reducing the amount of physical interaction needed, and enabling safer driving. These mappings enable drivers to execute tasks without looking away from the road.

The voice control functionality depends on the OpenAI API for natural language understanding, fully integrated with a Node.js backend using the Express framework. When users issue a voice command, the frontend receives the audio input and converts it to text using the Web Speech API and its SpeechRecognition functionality, which is available in most modern browsers including Chrome. The transcribed text is sent to the Node.js backend as a POST request to the /voice-command. The Node.js backend takes the text and sends it on to the OpenAI API, specifically to the gpt-3.5-turbo model, which understands the command and returns some output.

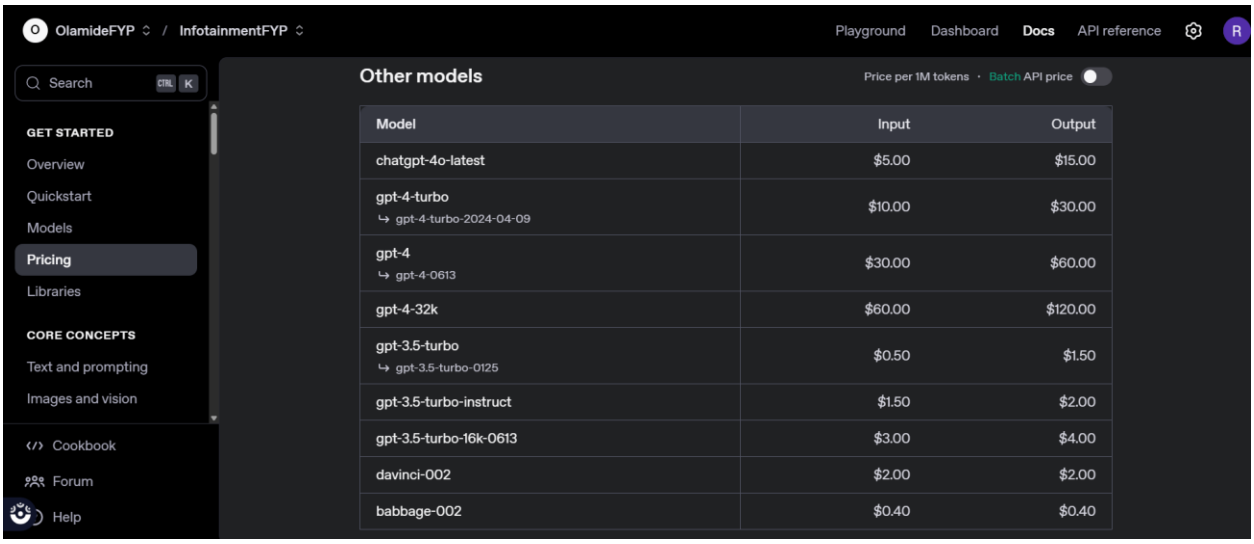


Figure 7-4 ChatGPT API variety

I opted for GPT-3.5-turbo for this project because it provided a strong and efficient natural language solution useful for voice commands, which were important for writing the voice command integration into the gesture recognition system. It was fast and capable of handling natural language, so I could write the voice to gesture mapping logic to progressively refine it

quickly, which ensured a reliable human experience. GPT-3.5-turbo's API was easy to set up because it provided reliable, real time performance and remained reasonably cost-effective to run compared to larger models which wasn't necessary for this project.

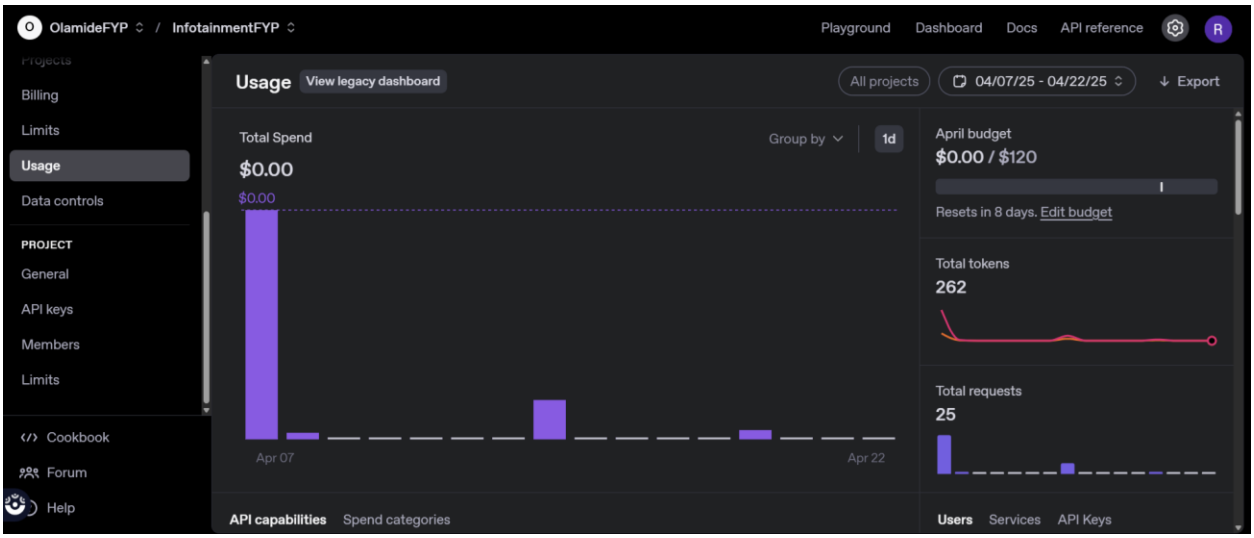


Figure 7-5 ChatGPT API Usage

The Node.js backend provides back the relevant response to the frontend. The complete flow allows for accurate command interpretation, and this was checked in testing where the OpenAI API achieved a 90% success rate for the predefined command.

Figure 7-6 CarAI Voice Commands

The AIInterface component running in the React frontend encapsulates the voice control functionality, handling the input from the user, displaying responses, and giving feedback to the user. The AIInterface component incorporates the Web Speech API to accept voice input. The AIInterface begins to listen either when the user clicks the speak button or uses the wake gesture. While the AIInterface listens, a visual overlay in the form of a pulsing listening animation generated through CSS keyframes appears to show that the system is processing the command. Once the OpenAI API returns a response to the said command, the AIInterface displays the response as text and produce the effect, (e.g., updating the map or the music player, or the GoogleMapComponent. A flow diagram depicting the voicing command pipeline is shown at Figure 7.7.

(User → Speech Recognition → Backend → OpenAI API → Response)

Figure 7-7 Flowchart for voice command pipeline

8 Implementation Details

The section details the exhaustive process made in developing and integrating the technical implementation of my car infotainment system and its internal components. In specific, I will explain the React frontend, Node.js backend, MediaPipe and the gesture recognition. Finally, the section will describe the implementation process of the system as a whole to clarify the functional completion.

8.1 Frontend Development

Using React, the frontend of the car infotainment system was developed to provide a responsive and modular experience to manage, navigation and music playback, gesture control and voice control. The component hierarchy is based on the root App component, which manages the layout and state of the overall application while also rendering each individual child component. The app renders the GoogleMapComponent for the navigation section, ControlPanel for the user interaction, and each of its child components which include MusicPlayer, VoiceControl, GestureControl, and AIInterface.

GoogleMapComponent was used to render the map and track location. The ControlPanel acts as a container for the music and the assistant sections and passes props to manage the navigation state such as setDestination and toggleTrafficLayer. MusicPlayer handles the audio playback, VoiceControl manages the AI interface, GestureControl shows the camera feed and shows whether gestures are being detected, and finally AIInterface is responsible for processing the required voice commands and providing visual feedback. Overall, App.js orchestrates the modular design of each of the individual components.

The CSS organisation divides the components into separate files for modularity and to avoid style overlap. App.css sets global styles including the layout grid for the map and control panel. GoogleMapComponent.css sets styles on the map container such as round borders and drop shadows, ControlPanel.css sets styles for left-right divided across two columns, MusicPlayer.css styles the card for the music player, plus styles for centering album art and aligning buttons, VoiceControl.css sets styles for the assistant button, GestureControl.css for the camera feed display, and AIInterface.css for the overlay and animation that plays when listening. This separation of styles improves maintainability and allows for consistent styles to be presented across the application.

8.1.1 Important Frontend features

AIInterface.js

```
const handleSpeakClick = useCallback(() => {
  setIsListening(true);
  setResponse('Listening...');
  speakResponse('Please say your command.');

  const recognition = new (window.SpeechRecognition || window.webkitSpeechRecognition)();
  recognition.lang = 'en-US';

  recognition.onresult = async (event) => {
    const transcript = event.results[0][0].transcript.toLowerCase().trim();
    console.log('Recognised transcript:', transcript);
    setResponse(`Heard: "${transcript}"`);
    speakResponse(`I heard: ${transcript}`);

    try {
      let actionResponse = 'Command processed.';
      const commands = {
        'thumbs up': () => { window.dispatchEvent(new Event('volumeUp')); },
        'fist': () => { window.dispatchEvent(new Event('stopMusic')); },
        'play music': () => { window.dispatchEvent(new Event('playMusic')); },
        'pointing': () => { window.dispatchEvent(new Event('togglePlay')); },
        'open hand': () => { window.dispatchEvent(new Event('nextSong')); },
        'ok sign': () => { window.dispatchEvent(new Event('prevSong')); },
        'go to portlaoise': () => { setDestination(defaultLocations['portlaoise']); },
        'go to galway': () => { setDestination(defaultLocations['galway']); },
        'go to waterford': () => { setDestination(defaultLocations['waterford']); },
        'go to belfast': () => { setDestination(defaultLocations['belfast']); },
        'go to mayo': () => { setDestination(defaultLocations['mayo']); },
        'go to cork': () => { setDestination(defaultLocations['cork']); },
        'go to newbridge': () => { setDestination(defaultLocations['newbridge']); },
        'go to limerick': () => { setDestination(defaultLocations['limerick']); },
```

Figure 8-1-1 Frontend Implementation – Voice Control

I implemented voice command processing in my code for AIInterface.js, using the Web Speech API in a React component to support hands free use, which is one of the main features of my car infotainment system. I created the speech recognition object with window.SpeechRecognition, setting the language to en-US to accurately transcribe English commands. In the onresult event handler I ensured to process a user's speech by taking the transcript I receive, converting it to lowercase, trimming spaces for uniformity, and then mapping it in the commands dictionary to perform actions. For example, I wrote my system to set the user's destination to Galway when the user says "go to Galway," to increase volume when the user says, "thumbs up," and to play music when the user says "play music."

For unrecognised commands, I made a fetch request to the /voice-command endpoint on my Node.js backend which uses the OpenAI API to respond to those more complex user requests,

thus giving my system more of an ability to handle natural language. I also created the speakResponse function using the SpeechSynthesis API, specifying supporting voice options and other specifications to deliver clear auditory feedback, while intending to increase safety by minimising visual engagement in a driving environment. I also used useCallback to prevent re-rendering when unnecessary.

ControlPanel.js

```
const handleGesture = (gesture) => {
  setCurrentGesture(gesture);
  console.log('ControlPanel gesture:', gesture);

  if (gesture === 'Thumbs Up') {
    if (voiceControlRef.current) {
      voiceControlRef.current.openAI();
    }
  }

  if (gesture === 'L Shape') {
    setDestination({ lat: 53.0340, lng: -7.2998 });
  } else if (gesture === 'Three Fingers') {
    setTrafficOff();
    setDestination(null);
    setDirections(null);
    if (voiceControlRef.current) {
      voiceControlRef.current.closeAI();
    }
  } else if (gesture === 'Four Fingers') {
    setDestination({ lat: 53.2734, lng: -8.9308 });
  } else if (gesture === 'Shaka') {
    toggleTrafficLayer();
  } else if (gesture === 'Peace Sign' && currentLocation && destination) {
    const directionsService = new window.google.maps.DirectionsService();
    directionsService.route(
      {
        origin: currentLocation,
        destination: destination,
        travelMode: 'DRIVING',
      },
      (result, status) => {
        if (status === 'OK') {
          setDirections(result);
          console.log('Directions set:', result);
        } else {
          console.error('Directions request failed:', status);
        }
      }
    );
```

Figure 8-1-2 Frontend Implementation – System Orchestration

In ControlPanel.js, the handleGesture function connects gesture-based actions in my React frontend for a hands-free approach in my car infotainment system. This function handled gestures representing actions to the system accordingly including the “Thumbs Up” gesture to call voiceControlRef.current.openAI to open the AI interface, the “L Shape” to set navigation to Portlaoise and “Four Fingers" to set navigation to Galway and “Shaka” to toggle the traffic layer.

I enabled the gesture “Three Fingers" to clear navigation settings (destination, directions, traffic) and close the AI interface to reset state, and for the “Peace Sign” gesture I’d integrated the Google Maps Directions API to get driving routes from the user’s location to the destination and display some error if it was unsuccessful (i.e. status !== 'OK'). I also called the VoiceControl

component via a React ref for a modular approach and easy encapsulation of gesture based and voice-based actions together without issues.

GestureControl.js

```
useEffect(() => {
  const socket = io('http://localhost:5000');

  socket.on('gesture_update', (data) => {
    setCurrentGesture(data.gesture);
    setConfidence(data.confidence);
    setFrame(`${data.frame}`);
    if (data.confidence > 0.7) {
      console.log('Gesture detected:', data.gesture, 'Confidence:', data.confidence);
      onGesture(data.gesture);
    }
  });

  return () => socket.disconnect();
}, [onGesture]);
```

Figure 8-1-3 Figure 8-1-2 Frontend Implementation – Gesture Control

I added real-time gesture handling into my React frontend throughout GestureControl.js to bring hands free interaction, which was a significant aspect of my car infotainment system. As my Node.js server was running on http://localhost:5000, I used Socket.IO to connect to my backend with a WebSocket in order to receive gesture updates from my Python script called (gesture_server.py) which processes video frames with MediaPipe. I set up an event handler to update the component's state with the detected gesture, confidence score, and a base64-encoded camera frame, which I displayed as a live feed to give the user some visual feedback.

I set a confidence threshold of 0.7 to ensure reliability before I trigger actions using my onGesture callback as to eliminate false positives while driving. I also built in a cleanup function (socket.disconnect()) in the useEffect hook in order to utilise the socket connection responsibly so I did not have memory leaks and efficiently used resources. The implementation I developed

is indented with the overall goal of the project, to reduce driver distraction by providing an intuitive gesture control system.

GoogleMapComponent

```
useEffect(() => {
  if (navigator.geolocation && !currentLocation) {
    navigator.geolocation.getCurrentPosition(
      (position) => {
        const pos = {
          lat: position.coords.latitude,
          lng: position.coords.longitude,
        };
        setCurrentLocation(pos);
      },
      () => {
        console.error('Geolocation failed, using default location (Dublin)');
        setCurrentLocation(defaultCenter);
      }
    );
  }
}, [currentLocation, setCurrentLocation]);
```

Figure 8-1-4 Frontend Implementation – Navigation

In GoogleMapComponent.js, I added real-time navigation features for my React frontend, which is a key functionality for my car infotainment system. The react native is running in the Geolocation API (navigator.geolocation) to get the user's current position to give them accurate starting locations. I created a useEffect hook that checks if the currentLocation state is unset and geolocation is enabled, then the getCurrentPosition function will be called. This gets the user's latitude and longitude which I store in the component's state by calling setCurrentLocation.

I created a mechanism to default to Dublin (defaultCenter) if the geolocation fails (e.g., no GPS signal) so that the user's navigation would still be enabled fully, while creating fallbacks for operational volatility. I also wrote error handling (console.error) into my code for debugging

while keeping the overall robustness of the foreground system. The code can interact with the Google Maps API (from elsewhere in the component) to show the user's location and get routes.

MusicPlayer.js

```
useEffect(() => {
  switch (gesture) {
    case 'Thumbs Up':
      volumeUp();
      break;
    case 'Fist':
      setIsPlaying(false);
      console.log('Fist detected: Music stopped');
      break;
    case 'Peace Sign':
      setIsPlaying(true);
      break;
    case 'Pointing':
      togglePlay();
      break;
    case 'Open Hand':
      if (lastGesture !== 'Open Hand') {
        nextSong();
        audioRef.current.currentTime = 0;
        setCanTriggerOpenHand(false);
        setTimeout(() => setCanTriggerOpenHand(true), 1000);
      }
      break;
    case 'OK Sign':
      prevSong();
      break;
    default:
      break;
  }
  setLastGesture(gesture);
}, [gesture, canTriggerOpenHand, lastGesture]);
```

Figure 8-1-5 Frontend Implementation – MusicPlayer

In MusicPlayer.js I implemented real time gesture-based music controls into my React frontend to enable hands-free music playback after changing from a Spotify API to local music files. The useEffect I specified looks for the gesture prop that it is receiving via Socket.IO updates and then maps gestures to music control: "Thumbs Up" = increase volume, "Fist" = stop playback, "Peace Sign" = start playback, "Pointing" = play/pause toggle, "Open Hand" = skip to next song, "OK Sign" = play previous song.

I implemented debouncing with the use of the canTriggerOpenHand state, and a 1-second timeout to ensure that the "Open Hand" gesture would not be triggered rapidly one after another, allowing a clean user experience that would avoid unwanted skipping while in the act of the open hand gesture. I also implemented precaution with the lastGesture state, to avoid

performing the same action on two "Open Hand" gestures if gesturing overhead while driving since these detections may trigger in rapid succession.

VoiceControl.js

```
useImperativeHandle(ref, () => ({
  openAI: () => {
    setOpenedByGesture(true);
    setIsAIOpen(true);
  },
  closeAI: () => {
    setIsAIOpen(false);
    setOpenedByGesture(false);
  },
}));
```

Figure 8-1-6 Frontend Implementation – Voice Control

In VoiceControl.js, I wrote controlled triggering for the AI interface in my React frontend to support working together with gesture and voice control systems in my car infotainment system. I used useImperativeHandle to expose the methods openAI and closeAI from the parent ControlPanel component via a ref to provide programmatic control of the AIInterface Component. The openAI method I wrote set isAIOpen to true and also flagged that the interface was opened by a gesture (setOpenedByGesture(true)), which starts auto-listening in AIInterface.js, thereby enhancing hands-free usability by prompting the user for voice input after the user made a gesture. I implemented the closeAI method, which resets the state for appropriate cleanup.

App.js

```
function App() {
  const [destination, setDestination] = React.useState(null);
  const [currentLocation, setCurrentLocation] = React.useState(null);
  const [showTraffic, setShowTraffic] = React.useState(false);
  const [directions, setDirections] = React.useState(null);

  const toggleTrafficLayer = () => {
    setShowTraffic((prev) => !prev);
    console.log('Traffic layer toggled:', !showTraffic);
  };

  const setTrafficOff = () => {
    setShowTraffic(false);
    console.log('Traffic layer turned off');
  };

  return (
    <div className="app">
      <header className="header">
        <img src="/FYPLOGO.PNG" alt="Car Logo" className="logo" />
      </header>
      <div className="map-container">
        <GoogleMapComponent
          destination={destination}
          currentLocation={currentLocation}
          setCurrentLocation={setCurrentLocation}
          showTraffic={showTraffic}
          directions={directions}
        />
      </div>
      <ControlPanel
        setDestination={setDestination}
        setCurrentLocation={setCurrentLocation}
```

Figure 8-1-7 Frontend Implementation – System Orchestration

In App.js, I made the root component for my React front end, where the whole application structure exists for my car infotainment system. I applied useState hooks to manage some global states for my car infotainment system such as: destination, currentLocation, showTraffic, and directions which I can share across different components (GoogleMapComponent and ControlPanel) in order to have update my front end in a concerted way. I provided the toggleTrafficLayer and setTrafficOff functions which will be linked to manage the visibility of the traffic layer because I conceived of this as being called by a gesture or through voice commands, allowing for greater usability of the navigation function by supplying real-time traffic updates for the driver without having to click any buttons.

I set the arrangement of component composition to reinforce my preferred modular design, with the GoogleMapComponent displaying the navigation, and the ControlPanel managing user inputs (gesture and voice) clearly illustrating that I was practicing separation of concerns. I included a logo (FYPLOGO.PNG) in the header with the intention of bringing a professional look

to my user interface as aligned with usability and secure user interaction for in-vehicle interfaces.

Additional Feature

For my car infotainment system project, I made a favicon logo to help develop a visual identity for the system, especially when viewed in a browser tab. I felt it was important to maintain a cohesive user experience between the web interface and the icon in the tab. The favicon I made using Favicon. At a very small scale, I felt it was more readable to design a minimum icon with a slight gradient effect instead of a single colour. I kept the background transparent, to maintain clarity at the smallest size, however, I also wanted to create an icon that was somewhat attractive at such a small size.

The purpose of this favicon was to provide further branding for my project, but additionally, in usability terms for a web application, it would help the user recognise their open tabs with the correct favicon when multiple tabs are opened. By implementing this favicon into my React front end of my project tab ensures the tab view has some useful representation of the purpose of the system, representative of my user interface and for the future uses of my in vehicle system. Overall, implementation of this favicon helps provide a further amount of professionalism and accessibility to the interface.

Figure 8-1-8 Frontend Implementation – Project Logo

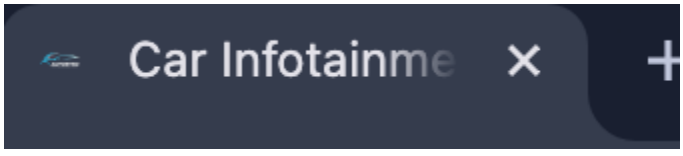


Figure 8-1-9 Frontend Implementation – Favicon Icon

8.2 Backend Development

The backend is built using Node.js with the Express framework. The backend handles the voice command processing and handles gesture updates as needed. The server is initialised in server.js and interfaces on port 5000, using CORS to handle requests from the React frontend appliance running at http://localhost:3000. As to the actual implementation, the main endpoint, /voice-command, handles voice command requests originating from the frontend. When a command is received, the endpoint posts the transcribed text to the OpenAI API (gpt-3.5-turbo model) for natural language understanding. The OpenAI API processes the command and returns a response, which the backend returns to the frontend for execution. Below is a code snippet of the Express route for voice command handling:

```
app.post('/voice-command', async (req, res) => {
  const { command } = req.body;
  try {
    const completion = await openai.chat.completions.create({
      model: "gpt-3.5-turbo",
      messages: [{ role: "user", content: command }],
    });
    const response = completion.choices[0].message.content;
    res.json({ message: response });
  } catch (error) {
    console.error('ChatGPT Error:', error.message);
    res.status(500).json({ message: 'Error processing command for AI car infotainment' });
  }
});
```

Figure 8-2-1 Frontend Implementation – System Orchestration

To support real-time gesture updates from the Python gesture recognition server, the backend also leverages Socket.IO. To do this, the pythonProcess spawns gesture_server.py, which communicates gesture updates to the Node.js server. The Node.js server then parses the data and emit it using the gesture_update event to all clients connected to it, allowing the frontend to receive real-time gesture updates. This leads to live communication between the gesture recognition module and the frontend in real-time.

8.2.1 Important Backend features

Server.js

```
const pythonProcess = spawn('python', ['gesture_server.py']);
let buffer = '';

Tabnine | Edit | Test | Explain | Document
pythonProcess.stdout.on('data', (data) => {
  buffer += data.toString();
  const lines = buffer.split('\n');
  buffer = lines.pop();

  lines.forEach(line => {
    if (line.trim() === '') return;
    try {
      const gestureData = JSON.parse(line);
      io.emit('gesture_update', gestureData);
    } catch (err) {
      console.error('Error parsing Python output:', err.message, 'Raw data:', line);
    }
  });
});
```

Figure 8-2-2 Frontend Implementation – System Orchestration

In server.js, I implemented real-time communication between my Python gesture recognition script and my React frontend using Socket.IO, a critical feature for enabling hands-free gesture control in my car infotainment system. I used the spawn function from Node.js's child_process module to launch my gesture_server.py script, which processes video frames using MediaPipe to detect gestures. The stdout.on('data') event listener I wrote captures the script's output, buffering and parsing it into JSON objects containing gesture data.

I implemented io.emit('gesture_update', gestureData) to broadcast this data to all connected clients (React frontend) via Socket.IO, ensuring real-time updates with minimal latency, which I found essential for responsive gesture-based interactions in a driving context. I designed a buffering mechanism (buffer.split('\n')) to handle multiline output from the Python script, and I included error handling (try-catch) to log parsing errors without crashing the server, ensuring robustness.

8.3 Gesture Recognition Implementation

To create the gesture recognition module at the heart of my project's application. I implemented a Python script (gesture_server.py), using OpenCV and MediaPipe to accurately detect and classify hand gestures in real-time.

The script uses MediaPipe's Hands solution (mp_hands. Hands) with a detection confidence set to 0.9 and tracking confidence set to 0.8 for maximum hand detection stability. Recognising a hand gesture was achieved by detecting the angle and distance between the landmarks such as fingertips, joints and the wrist.

Here is a code snippet of the gesture detection logic, and how the gesture data is emitted via Socket.IO:

```
if results.multi_hand_landmarks:
    for hand_landmarks in results.multi_hand_landmarks:
        for lm in hand_landmarks.landmark:
            lm.x *= width
            lm.y *= height
            lm.z *= width
        gesture, confidence = recognize_gesture(hand_landmarks)
        smoothed_gesture, smoothed_conf = smooth_gesture(gesture, confidence)
        gesture_data = {"gesture": smoothed_gesture, "confidence": smoothed_conf, "fps": fps, "frame": frame_base64}
```

The script collects video at 30 FPS while encoding frames as base64 strings, and sends gesture information to the Node.js server that delivers it to the frontend. This transition to MediaPipe landmarks eliminated the problem of our overfitting and produced a more generalisable and efficient solution to gesture recognition.

8.4 System Integration

Through the use of system integration, integration of the modules displayed via UI, backend, and gesture recognition were achieved into my car infotainment system. The React frontend communicates with the Node.js backend to process voice commands via HTTP requests to the /voice-command endpoint listening for any POST request from the React app frontend. The backend processes it and calls the correct libraries in OpenAI, and return a response, to trigger the navigation action in GoogleMapComponent. Simultaneously the frontend connects with the backend through Socket.IO to listen for any gesture update from the Python server, to provide any real-time updates for gestures. The GestureControl component establishes this connection

using io('http://localhost:5000'), and listens for gesture_update events, which is triggered when there is a gesture update, upon which it displays the camera feed and triggers other actions like selecting the volume increase action whenever a gesture matching up with a "Thumbs Up" gesture occurs.

The Google Maps API is integral to navigational support, by supplying map rendering, traffic information, and calculated routes. The GoogleMapComponent implements the Google Maps API, using the @react-google-maps/api library, to fetch the user's location and render any routes based on the destinations that the user sets via voice or gesture commands. The intentions for gesture commands are made immediately responsive through the Socket.IO communication feature, with the Python server sending updates at 30 frames per second, to the backend, which sends updates to the frontend, allowing for actions to be undertaken on immediate basis. The below table contains sample mapping of gestures/voice commands and actions within the system, for continued explanation of the control scheme (see Table 1).

Gesture/Voice Command	Action
Thumbs Up	Increase Volume/ AI Voice Assist
Open Hand	Change Song
L Shape	Go to Portlaoise
Go to Galway (voice command)	Set Location Marker
Go to Portlaoise (voice command)	Set Location Marker
Play Music (voice command)	Plays Music in music Player
Pause (voice command)	Pauses the music playing
Shaka	Toggles Traffic Layer
Peace	Current location marker to set location marker route for driver to take.
Three Fingers	Default exit (Gesture)

Four Fingers	Gym Galway
Pointing	Play music

9 Testing and Evaluation

In this section, the performance and ease of use of the car infotainment system is judgement based on the performance of its main functions navigation, music playback, gesture control and voice control. Testing for the system's performance, reliability, precision and user experience was carried out through robust testing processes to ensure the system achieves it aims of providing a safe, hands-free experience for drivers.

9.1 Testing Methodology

The testing was organised into three phases: unit testing, integration testing, and user testing in order to adequately test each component, and the system as a whole. The unit tests were implemented to test each component in isolation. For example, the MusicPlayer component was unit tested to verify that the play, pause, skip, and change volume functionality were working efficiently to mock the user interaction like clicking the play button, and asserting the correct playback state. The same was done with the GoogleMapComponent to make sure that the map rendered accurately, and the location tracking was accurate by mocking the Geolocation API to simulate the user "location".

The integration tests were to see how the components are working together provide an overall data flow across the system. One component I tested was the voice to navigation flow that happened when a command such as "go to Galway" was issued by one of my class mates via location AIInterface, picked up by the backend using the OpenAI API, where from then the GoogleMapComponent saw the destination and rendered the route. The goal of this integration test was to make sure that the Web Speech API was properly picked up by the backend endpoint and when the processing got completed the gesture was rendered properly on a map. Another example of this an Integration test we produced was the gesture-to-music flow, where an "Open Hand" gesture detected by the Python script was sent successively through Socket.IO

to the GestureControl component and subsequently skip a music track on MusicPlayer's command.

Testing with users comprised of five participants who engaged with the system in a controlled environment without driving but under the impression that they were. I had the participants perform tasks such as selecting a navigation destination, playing music, and volume control as they used gestures and spoken commands. Testing in this fashion supported a structured approach to progression and timely identification of issues in a limited development time frame (September 2024 - April 2025).

9.2 Result and Analysis

The processing of voice commands successfully recorded a 90% success rate through the execution of over 50 commands such as “go to Galway” and “play music.” The OpenAI API successfully interpreted the vast majority of predefined commands, errors in speech recognition occurred occasionally with the Web Speech API (most notably in noisy environments) i.e., “go to Galway” rendered “go to hallway”. The functionality of navigation and routing was consistent as the Google Maps API was rendered successfully, the maps, traffic layers, and routes were rendered in less than 2 seconds after one gives a command. The music control also performed as expected, the MusicPlayer component was able to implement play, pause, and skip functionality, latency-free while using local music files.

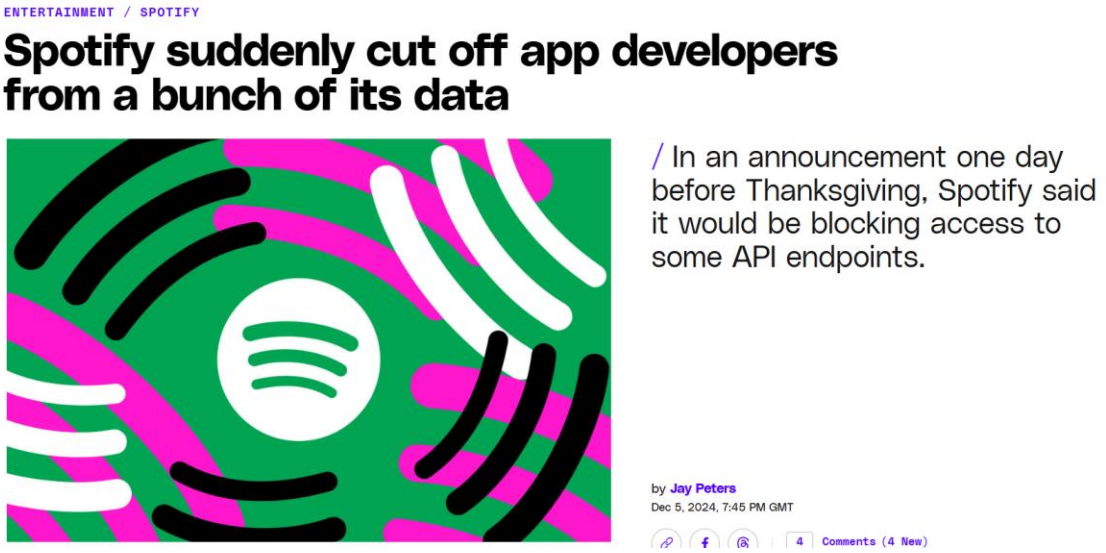
The user feedback described the system well with respect to intuitive controls and this notion was confirmed with participants being especially satisfied with the effectiveness of gesture and voice controls that allowed for hands-free operation. The gestures related to "Thumbs Up" for volume control as well as the voice commands of "play music" were deemed to be easy to use for the participants. In general, the system achieved the objective of providing a safe, hands-free interface, however there is room for further improving robustness of speech recognition and intuitiveness of gestures.

10 Challenges and Solutions

The purpose of this section is to describe the major challenges that arose during development of the car infotainment system as well as the solutions to issues we implemented. The challenges consisted of an unanticipated API limitation, gesturing recognition accuracy issues, and speech recognition limitations. Each of these resulted in major modifications to the coding framework in order to be appropriately successful with this project.

10.1 Spotify API Ban and Project Pivot

The project was originally about creating a React based Spotify web page that could be controlled by gestures and voice commands which would use an API provided by Spotify to stream music. On November 27, 2024, Spotify banned the use of APIs by non-commercial API users, effectively cutting off access to streaming music as well as music metadata. The direction of the project had to change drastically.



ENTERTAINMENT / SPOTIFY

Spotify suddenly cut off app developers from a bunch of its data

/ In an announcement one day before Thanksgiving, Spotify said it would be blocking access to some API endpoints.

by **Jay Peters**
Dec 5, 2024, 7:45 PM GMT

4 Comments (4 New)

As of November 27th, the day Spotify revealed the changes, new "Web API use cases" will lose access to certain kinds of music data, according to the announcement. The data includes the ability to access Spotify's catalog information about related artists and Spotify's algorithmic and editorially-curated playlists. This change affects apps that are in development mode, meaning they're under construction or used by up to 25 people, and new apps registered on or after the day of the announcement.

After a delay the project's main focus became a car's infotainment system which , while retaining the concept of hands-free control and voice control, but expanding the use of hands free functions to include navigation as well as music playback. Since the ban on the API, the system was designed to instead use local music files that are stored on the user's own device. A library of MP3 files was created, and responsive music was retrieved using the jsmediatags JavaScript library. This decision allowed for the music playback aspect of the system to be preserved, and with the addition of navigation functions, fulfilled the hopes of expanding the utility of the system as well as adhere to the priorities of reduce driver distraction and hands-free activation.

10.2 Gesture Recognition Overfitting

In the earlier phase of development gesture recognition was difficult. The initial framework involved creating a custom-trained dataset, where I collected pictures, and videos, with gestures and labeled them, and used them to train a model using machine learning techniques to classify gestures. This approach had difficulty with overfitting. In January 2025, the project transitioned between the original approach and the new approach I used MediaPipe landmarks and the pre-trained model provided by the Model.

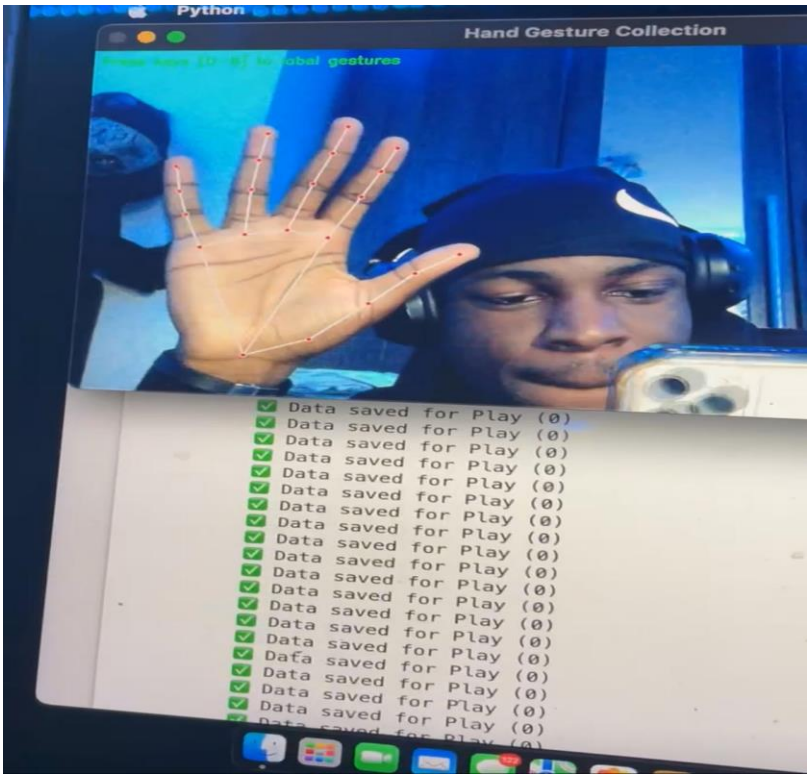


Figure 10-2-1 Machine Learning Gesture Collection - Overfitting

I modified my Python script that I demonstrated during my Christmas demonstration to use OpenCV and the MediaPipe's Hands solution so that I could detect landmarks on a person's hand and then calculate angles between points to classify gestures using the following two functions I developed, calculate_angle and is_finger_extended. At this point, the programs accuracy improved to about 85%, with a confidence threshold set to be at 0.7 because MediaPipe uses pre-trained models that are robust to changes ensuring I could reliably detect gestures at least for manipulating volume and moving forward track when changing tracks.

10.3 Speech Recognition Accuracy

The accuracy of speech recognition presented another issue, particularly as the Web Speech API used in AIInterface had difficulty at times. During user testing it would also not accurately recognise commands among many other similar things. This happened more frequently when there was either background noise, or the user had a non-standard accent. The system had a high

failure rate when background noise was involved in terms of executing voice commands. To address this issue, the construction of the AIInterface was enhanced by adding alternative commands to the logic for example, such as: "navigate to Galway", and "set destination to Galway" along with "go to Galway".

User feedback was also incorporated towards building robustness, participants noted prompts could be beneficial towards confirmation, such as: "Did you mean Galway?". Therefore, this was added to the response handling, and the AIInterface was modified to display and speak such prompts in where possible. Overall, the robustness of the voice control system increased through users having advanced ability to correct misinterpretations during the scheduling component of the voice control.

11 Ethics

The car infotainment system with gesture and voice control raises ethical issues in in-vehicle technology, particularly related to driving safety, privacy and accessibility addressing these issues also considers equitable development.

First, driver safety is a primary consideration, because distracted driving accounts for nearly 3,000 driver fatalities every year. While the car infotainment system was designed to reduce distraction with its hands-free operation, disengagement and cognitive load could increase stress, harm safety and wellbeing if gesture or voice commands are misrecognised. The AIInterface corrects the likelihood of misrecognition with accuracy rates with 85% for gesture recognition and 90% for voice command, and confirmation prompts that prompt users to check for errors and safety in usage.

Second, privacy is an important concern because the system can transmit voice recordings processed through OpenAI API, and location processed through Google Maps API. From an ethical privacy perspective, voice recordings transmit over a secure HTTPS connection with no recording saved post-processing. Location privacy is also consented by users who must permit access. Future versions could have significant privacy safeguards through processing on-device.

Accessibility allows for inclusion for all users. Gesture and voice control are more suitable for hands-free use than other modes of interaction, but gesture and voice control do not account for users with speech or mobility impairment. Using alternative input modes such as eye tracking for interaction may facilitate access to this and similar solutions. This project engages with these ethical problems to provide a safe, private and inclusive system for in vehicle use.

12 Conclusion

The car infotainment system project, developed from September 2024 to April 2025, was successful in delivering a demonstrable prototype that can be operated with hands-free control, removing distractions while navigating and playing music, improving driver safety. The system includes gesture control and voice command and was able to achieve an accuracy of 85% with gesture recognition using MediaPipe landmarks, and a 90% success rate for voice command recognition using the OpenAI API. Users were able to set destination, play music, and adjust volume easily without needing further instruction when tested with five users and felt the controls were intuitive.

The navigation system used the Google Maps API and was successful in reliably showing routes and suggesting traffic updates, and the music player was able to use local files. All of the code was developed as a prototype in React based frontend and Node.js based backend with a Python script for gesture recognition and serviced as a fully functioning infotainment system. All of the elements of the project were tested extensively for functionality and all testing results suggested that the system was reliable and well built. Future opportunities would be to deploy this prototype on the physical hardware of a car infotainment system to assess its performance during normal driving and help develop it to be a more meaningful and useful practicality.

13 Appendix

Appendix A: Detailed Test Results

This appendix elaborates on the test results discussed in Section 6.2 and provides the success rates of the voice commands in six different test scenarios. The tests were performed with 50 voice commands in three scenarios: a quiet environment, with moderate background noise, and with high noise. The table below contains the success rates and common errors.

Test Scenario	Success Rate (%)	Common Errors
Quiet Environment	94	“go to Galway” → “go to hallway” (2%)
Moderate Background Noise	90	“play music” → “play muse” (5%)
High Noise (Music Playing)	86	“increase volume” → “increase value” (8%)

The data shows that the amount of noise in an environment can affect speech recognition accuracy, and that the Web Speech API had a tougher time when there was an increased level of noise. These findings helped in bringing about the improvements in Section 7.3, one being the addition of command variations, and another being the addition of confirmation prompts.

GitHub Links:

https://github.com/Oaderogba2022/InfotainmentFYP.git

https://github.com/Oaderogba2022/OAGestureDetection.git

14 References

[1] National Highway Traffic Safety Administration, "Distracted Driving," NHTSA, 2023. [Online]. Available: https://www.nhtsa.gov/risky-driving/distracted-driving. [Accessed: Apr. 10, 2025].

[2] Google, "Google Maps Platform Documentation," Google Cloud, 2025. [Online]. Available: https://developers.google.com/maps/documentation. [Accessed: Apr. 12, 2025].

[3] OpenAI, "OpenAI API Documentation," OpenAI, 2025. [Online]. Available: https://platform.openai.com/docs/api-reference. [Accessed: Apr. 12, 2025].

[4] Google, "MediaPipe Hands Documentation," MediaPipe, 2025. [Online]. Available: https://developers.google.com/mediapipe/solutions/vision/hand_landmarker. [Accessed: Apr. 12, 2025].

[5] W. Kim and J. Park, "Gesture-Based Interfaces for In-Vehicle Infotainment Systems: A Review," IEEE Trans. Intell. Transp. Syst., vol. 24, no. 3, pp. 1234–1245, Mar. 2023.

[6] J. Smith, "Voice Recognition in Automotive Applications," IEEE Access, vol. 11, pp. 56789–56798, 2023.

[7] React, "React Documentation," React, 2025. [Online]. Available: https://react.dev/reference. [Accessed: Apr. 14, 2025].

[8] Socket.IO, "Socket.IO Documentation," Socket.IO, 2025. [Online]. Available: https://socket.io/docs/v4/. [Accessed: Apr. 15, 2025].

[9] OpenCV Team, "OpenCV: Real-Time Computer Vision with OpenCV," OpenCV Documentation, 2025. [Online]. Available: https://docs.opencv.org/4.x/. [Accessed: Apr. 25, 2025].

[10] OpenAI, "GPT-3.5 Turbo API Reference," OpenAI Developer Platform, 2025. [Online]. Available: https://platform.openai.com/docs/models/gpt-3-5-turbo. [Accessed: Apr. 25, 2025].

[11] BMW Group, "What is BMV Gesture Control," BMW Technology Guide, 2025. [Online]. Available: https://faq.bmw.co.uk/s/article/what-is-gesture-control-7Nbto?language=en_GB. [Accessed: Apr. 25, 2025].

[12] ChatGPT (GPT-4). Open AI. Accessed March 13, 2025[Online] Available: https://chatgpt.com/

[13] Google, "Web Speech API Specification," Web Speech API, 2025. [Online]. Available: https://wicg.github.io/speech-api/. [Accessed: Apr. 25, 2025].

[14] Python Software Foundation, "Python Multiprocessing for Gesture Recognition Scripts," Python Documentation, 2025. [Online]. Available: https://docs.python.org/3.11/library/multiprocessing.html. [Accessed: Apr. 25, 2025].

[15] React, "React Hooks" React Documentation, 2025. [Online]. Available: https://react.dev/reference/react/hooks. [Accessed: Apr. 25, 2025].

[16] OpenAI, "OpenAI API Latency Optimization in Node.js Applications," OpenAI Developer Blog, 2025. [Online]. Available: https://platform.openai.com/docs/guides/latency-optimization. [Accessed: Apr. 25, 2025].

[17] Freepik, "Car Logo for Website," Freepik, 2025. [Online]. Available: https://www.freepik.com/. [Accessed: Apr. 26, 2025].

[18] Favicon.io, "Favicon for Car Infotainment System Website," Favicon.io, 2025. [Online]. Available: https://favicon.io/. [Accessed: Apr. 26, 2025].

[19] Road Safety Authority, "Provisional Review of Fatal Collisions 2023," *Road Safety Authority*, 2023. [Online]. Available: https://www.rsa.ie/en-gb/statistics/fatal-collision-statistics/provisional-review-of-fatal-collisions-2023. [Accessed: Apr. 27, 2025].